

# User Roles & Permissions

Last updated by | Haroun Ahmady | Apr 14, 2026 at 9:08 AM PDT

## Roles & Permissions

BankClient360 uses a hybrid **RBAC + ABAC** (Role-Based + Attribute-Based Access Control) system with numeric privilege levels, audit logging, and SAML integration.

### Roles

Roles are managed in the database via the Admin > User Management page. Each role is assigned a **privilege level** (0-4) that determines its authority tier. The following roles are configured in the system:

| Role                 | Privilege Level | Description                                              |
|----------------------|-----------------|----------------------------------------------------------|
| System Admin         | 4               | Full system access. Manages users, roles, SAML mappings. |
| Branch Manager       | 3               | Branch-level authority. Can view employee customer data. |
| Assistant Manager    | 2               | Team management. Limited user management.                |
| Loan Officer         | 2               | Lending operations. Account and customer access.         |
| Business Banker      | 2               | Business relationship management.                        |
| Teller               | 1               | Day-to-day customer and account operations.              |
| Customer Service Rep | 1               | Basic customer lookup and service.                       |
| Risk Analyst         | 1               | Risk monitoring and review.                              |
| Compliance Officer   | 1               | Compliance monitoring.                                   |

**Note:** Roles and their privilege levels are database-managed. The list above reflects the current deployment. New roles can be created via the Admin UI or directly in the `role` table.

### Privilege Levels

Privilege levels are **hierarchical** — a user at level 3 automatically inherits all permissions that require level 3 or below.

| Level | Tier          | Authority                                                         |
|-------|---------------|-------------------------------------------------------------------|
| 4     | System Admin  | Everything. User/role management, SAML config, all data.          |
| 3     | Senior/Branch | Can view employee customer records. Most operational access.      |
| 2     | Manager       | Team management. Can view user list. Standard operational access. |
| 1     | Staff         | Basic customer and account operations. No admin access.           |
| 0     | Read-Only     | Minimal access. View-only on permitted resources.                 |

## What Each Role Sees in the App

### Customer Dashboard Tabs

| UI Section                 | Permission Required     | Level 4 | Level 3 | Level 2 | Level 1 | Level 0 |
|----------------------------|-------------------------|---------|---------|---------|---------|---------|
| <b>Client tab</b>          | <i>(always visible)</i> | Yes     | Yes     | Yes     | Yes     | Yes     |
| <b>Household tab</b>       | household.view          | Yes     | Yes     | Yes     | *       | *       |
| <b>Accounts tab</b>        | accounts.view           | Yes     | Yes     | Yes     | *       | *       |
| <b>Account Summary tab</b> | accounts.view           | Yes     | Yes     | Yes     | *       | *       |

\* Depends on whether the role has this permission assigned in the `role_permission` table.

### Client Tab Sections

| UI Section                | Permission Required                | Description                       |
|---------------------------|------------------------------------|-----------------------------------|
| Relationship Summary card | customer.view.relationship_summary | Total relationship value overview |
| Recent Activity card      | customer.view.recent_activity      | Recent contact history            |
| Deposits section          | customer.view.deposits             | Deposit account details           |

If the user lacks the permission, the section is **hidden** (not shown at all).

### Account Table Columns

| Column        | Permission Required                | Description                            |
|---------------|------------------------------------|----------------------------------------|
| Balance       | <code>account.view.balances</code> | Current balance amount                 |
| Interest Rate | <code>account.view.balances</code> | Account interest rate                  |
| Total Balance | <code>account.view.balances</code> | Summary row with total across accounts |

Without `account.view.balances` , the user sees the account list but **balance and interest rate columns are hidden**.

### Employee Customer Protection (ABAC)

Bank employees can also be customers. When viewing a customer flagged as `isEmployee = true` , the system restricts access based on privilege level. The check is `maxPrivilegeLevel < 2` — meaning only **level 0 and level 1 users are restricted**.

| User Privilege Level     | Can See Accounts/Transactions? | Behavior                                                                                |
|--------------------------|--------------------------------|-----------------------------------------------------------------------------------------|
| Level 4 (System Admin)   | Yes                            | Full access to employee customer data                                                   |
| Level 3 (Branch Manager) | Yes                            | Full access to employee customer data                                                   |
| Level 2 (Manager)        | Yes                            | Full access to employee customer data                                                   |
| Level 1 (Teller)         | <b>No</b>                      | Accounts & Account Summary tabs hidden. Auto-redirected to Client tab. API returns 403. |
| Level 0 (Read-Only)      | <b>No</b>                      | Accounts & Account Summary tabs hidden. Auto-redirected to Client tab. API returns 403. |

This is enforced at **both layers**:

- **Frontend:** Accounts and Account Summary tabs are hidden; if already on those tabs, the user is auto-redirected to the Client tab.
- **Backend:** Transaction API endpoints build ABAC context by checking all account owners for `isEmployee === true` and deny access if privilege level is insufficient. Denials are audit-logged.

### Administration

| UI Section                   | Permission Required                       | Typical Roles                |
|------------------------------|-------------------------------------------|------------------------------|
| "User Management" menu item  | <code>users.view</code>                   | System Admin, Branch Manager |
| View user list & details     | <code>users.view</code>                   | System Admin, Branch Manager |
| Assign/remove roles          | <code>users.assign_roles</code>           | System Admin                 |
| View SAML role mappings      | <code>user_management.view</code>         | System Admin                 |
| Create/edit/delete SAML maps | <code>user_management.assign_roles</code> | System Admin                 |

### Permission Codes

Permissions follow a `resource.action` naming convention. A user gets permissions from two sources:

1. **Role-based** — permissions assigned to their role(s) in the `role_permission` table
2. **Privilege-level-based** — permissions where `minPrivilegeLevel <= user's max privilege level`

### Customer & Account Permissions

| Permission Code                                 | Description                            |
|-------------------------------------------------|----------------------------------------|
| <code>accounts.view</code>                      | View customer accounts and details     |
| <code>account.view.balances</code>              | View balance and interest rate columns |
| <code>transaction.view</code>                   | View transaction history               |
| <code>customer.view.relationship_summary</code> | View relationship summary card         |
| <code>customer.view.recent_activity</code>      | View recent activity card              |
| <code>customer.view.deposits</code>             | View deposits section                  |
| <code>household.view</code>                     | View household tab and relationships   |

### Administration Permissions

| Permission Code              | Description                               |
|------------------------------|-------------------------------------------|
| users.view                   | View user list, details, and roles list   |
| users.assign_roles           | Assign or remove roles from users         |
| user_management.view         | View SAML role mappings                   |
| user_management.assign_roles | Create, update, delete SAML role mappings |

### Protected API Routes

| Route                                   | Method | Required Permission          | Notes                               |
|-----------------------------------------|--------|------------------------------|-------------------------------------|
| /api/customers/:id/accounts             | GET    | accounts.view                | ABAC: checks customer context       |
| /api/accounts/:accountId/transactions   | GET    | transaction.view             | ABAC: checks employee-customer flag |
| /api/customers/:customerId/transactions | GET    | transaction.view             | ABAC: checks customer context       |
| /api/admin/users                        | GET    | users.view                   |                                     |
| /api/admin/users/:id                    | GET    | users.view                   |                                     |
| /api/admin/users/:id/roles              | POST   | users.assign_roles           |                                     |
| /api/admin/users/:id/roles/:roleId      | DELETE | users.assign_roles           |                                     |
| /api/admin/users/:id/roles/manual       | POST   | users.assign_roles           |                                     |
| /api/admin/roles                        | GET    | users.view                   |                                     |
| /api/admin/saml-mappings                | GET    | user_management.view         |                                     |
| /api/admin/saml-mappings                | POST   | user_management.assign_roles |                                     |
| /api/admin/saml-mappings/:id            | PATCH  | user_management.assign_roles |                                     |
| /api/admin/saml-mappings/:id            | DELETE | user_management.assign_roles |                                     |

## Attribute-Based Access Control (ABAC)

Some permissions support fine-grained, attribute-based checks beyond simple role membership. This is configured via `attributeConfig` on individual permissions.

### How It Works

1. A permission is marked `isAttributeBased = true` with an `attributeConfig` containing conditions.
2. When the permission is checked, a **context** is built from the request (e.g., the customer being accessed).
3. Each condition is evaluated against the context.
4. If a condition matches and `denyIfMatch = true`, access is denied — unless the user's privilege level meets the `minPrivilegeOverride` threshold.

### Condition Operators

| Operator                  | Description            |
|---------------------------|------------------------|
| <code>equals</code>       | Exact match            |
| <code>not_equals</code>   | Not equal              |
| <code>in</code>           | Value is in a list     |
| <code>not_in</code>       | Value is not in a list |
| <code>greater_than</code> | Numeric greater than   |
| <code>less_than</code>    | Numeric less than      |

### Example: Employee Customer Protection

```
Condition:
  attribute:      customer.isEmployee
  operator:      equals
  value:         true
  denyIfMatch:   true
  minPrivilegeOverride: 3
  reason:        "Employee customer records require elevated access"
```

### Available Context Objects

| Context Key | Type     | Description                 |
|-------------|----------|-----------------------------|
| customer    | Customer | The customer being accessed |
| account     | Account  | The account being accessed  |
| note        | Note     | The note being accessed     |

## How Permissions Are Resolved

When a request hits a protected route:

1. **Authentication** — `req.employeeId` must be set (401 if missing).
2. **Load permissions** — The user's active roles are fetched, along with all permissions granted by those roles and by the user's maximum privilege level.
3. **Privilege level check** — If `minPrivilegeLevel` is set on the middleware, the user's max privilege level is compared.
4. **Permission code check** — If `requireAll`, `requireAny`, Or `permissionCode` is set, the user's permission list is checked.
5. **ABAC evaluation** — If the permission is attribute-based, the context is built and conditions are evaluated.
6. **Audit logging** — Both grants and denials are emitted as audit events ( `AUTHZ_PERMISSION_GRANTED` / `AUTHZ_PERMISSION_DENIED` ).

Permission denials are also logged to the `permission_denial_log` table for compliance.

## Database Schema

### Core Tables

| Table                        | Purpose                                                    |
|------------------------------|------------------------------------------------------------|
| <code>privilege_level</code> | Defines numeric levels (0-4) with names                    |
| <code>role</code>            | Role definitions with privilege level and metadata         |
| <code>permission</code>      | Permission codes with optional ABAC config                 |
| <code>role_permission</code> | Maps roles to permissions (many-to-many)                   |
| <code>employee_role</code>   | Assigns roles to employees with effective/expiration dates |

### Audit & Compliance Tables

| Table                   | Purpose                               |
|-------------------------|---------------------------------------|
| role_audit_log          | Tracks all role assignment changes    |
| permission_denial_log   | Records every denied permission check |
| employee_status_history | Tracks employee status changes        |
| employee_role_history   | Full history of role assignments      |
| role_change_request     | Workflow for role change approvals    |

SAML Integration

| Table             | Purpose                                              |
|-------------------|------------------------------------------------------|
| saml_role_mapping | Auto-assigns roles based on SAML identity attributes |

Frontend Usage

Permission Hooks

```
import { useHasPermission, useHasAnyPermission, useMinPrivilegeLevel } from "@/hooks/usePermissions";

// Single permission check
const canViewUsers = useHasPermission("users.view");

// Any of multiple permissions
const canManage = useHasAnyPermission(["users.view", "users.assign_roles"]);

// Privilege level check
const isManager = useMinPrivilegeLevel(2);
```

PermissionGuard Component

Conditionally renders children based on permissions:

```
import { PermissionGuard } from "@/components/PermissionGuard";

<PermissionGuard permissionCode="users.assign_roles" fallback=<p>Access denied</p>>
  <RoleAssignmentForm />
</PermissionGuard>

<PermissionGuard minPrivilegeLevel={3}>
  <SeniorManagerDashboard />
</PermissionGuard>

<PermissionGuard requireAny=["users.view", "user_management.view"]>
  <AdminPanel />
</PermissionGuard>
```

---

## Role Testing (Non-Production)

The system includes a role testing feature that lets developers simulate different roles without changing actual assignments. Controlled by:

- **Environment variable:** `ROLE_TESTING_ENABLED` (defaults to `true`, set to `false` to disable)
- **Production guard:** Always disabled in `NODE_ENV=production`

When active, a user can temporarily override their role to see the application as that role would experience it. The override replaces (not merges) the user's permissions with the test role's permissions. A yellow banner appears at the top of the page when role testing is active.

---

## Key Files

| File                                        | Description                                 |
|---------------------------------------------|---------------------------------------------|
| shared/schema.ts                            | Type definitions and table schemas          |
| server/middleware/permissions.ts            | Express permission middleware               |
| server/services/permissionService.ts        | Core permission checking logic              |
| server/services/roleTestService.ts          | Role testing/override service               |
| server/storage/roleManagement/              | Data access layer (Postgres + SQL Server)   |
| client/src/hooks/usePermissions.ts          | React permission hooks                      |
| client/src/components/PermissionGuard.tsx   | Declarative permission guard component      |
| client/src/components/CustomerDashboard.tsx | Main dashboard with tab/section visibility  |
| client/src/components/AccountList.tsx       | Account table with column-level permissions |
| client/src/components/TopBar.tsx            | Navigation with admin menu visibility       |
| client/src/pages/UserManagement.tsx         | Admin page for user and role management     |